你說：

在 500×400 的畫布上畫出一幅「小房子」風景畫

自己規劃位置，練習座標思考

必要元素：
1.**天空與草地**：用 fillRect 把畫面上半部塗成天藍色、下半部塗成綠色
●
2.**太陽**：用 fillOval 畫黃色正圓，並用 drawLine 畫至少 8 條光芒（提示：可用 for 迴圈搭配 Math.cos、Math.sin）
●
3.**雲朵**：用至少 3 個互相重疊的白色 fillOval 組成
●
4.**房屋主體**：用 fillRect + drawRect
●
5.**屋頂**：只能用 drawLine 畫出三角形（三條線段）
●
6.**門**：用 fillRoundRect，門把用小的 fillOval
●
7.**窗戶**：至少 2 扇，用 fillRect + drawRect，並用 drawLine 畫十字窗框
●
8.用 g.drawString("學號 姓名", x, y) 在畫面角落簽名 我要寫這個作業 你可以迸我一個基本的程式框架嗎

ChatGPT 說：

可以！這題我們先不要急著把所有東西畫完，我先給你一個 **Java Swing 基本程式框架**，把畫畫的區域、座標系統和各個元素的位置安排好。

你之後只需要按照順序填入繪圖程式即可。

一、基本程式框架

```
import javax.swing.*;
import java.awt.*;

public class Main {

    public static void main(String[] args) {

        JFrame frame = new JFrame("小房子");

        MyPanel panel = new MyPanel();

        frame.add(panel);

        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

    }
}

class MyPanel extends JPanel {

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
```

```
        // 1. 天空與草地

        // 2. 太陽

        // 3. 雲朵

        // 4. 房屋主體

        // 5. 屋頂

        // 6. 門

        // 7. 窗戶

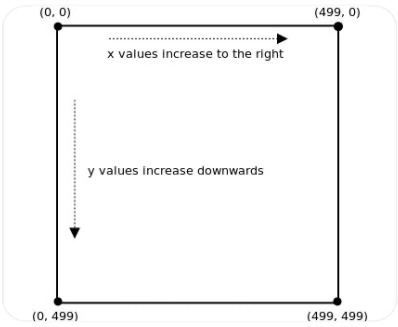
        // 8. 簽名
        g.drawString("學號 姓名", 400, 380);

    }
}
```

二、先理解畫布座標

你的畫布是 500 × 400。

Java Swing 的座標原點在左上角：



- X：往右增加
- Y：往下增加
- 左上角：(0, 0)
- 右上角：(500, 0)
- 左下角：(0, 400)
- 右下角：(500, 400)

三、建議的房屋配置

我們先規劃位置，避免畫出來的東西互相重疊。

元素	建議座標
天空	(0, 0)，寬500、高250
草地	(0, 250)，寬500、高150

太陽	(390, 40)，直徑60
雲朵	(60, 60) 附近
房屋主體	(170, 190)，寬180、高130
屋頂	三個頂點約 (150, 195)、(260, 110)、(370, 195)
門	(240, 250)，寬45、高70
左窗戶	(190, 220)，寬40、高40
右窗戶	(300, 220)，寬40、高40

四、你需要記住的繪圖方法

方法	用途
`g.setColor()`	設定顏色
`g.fillRect()`	填滿矩形
`g.drawRect()`	畫矩形外框
`g.fillOval()`	填滿橢圓或圓形
`g.drawOval()`	畫橢圓或圓形外框
`g.drawLine()`	畫線
`g.fillRoundRect()`	填滿圓角矩形
`g.drawString()`	顯示文字

五、我們的練習順序

我建議照這個順序寫，這樣比較容易理解每個方法。

1. 天空與草地：練習 `fillRect`
2. 太陽：練習 `fillOval` 、 `drawLine`
3. 雲朵：練習重疊的 `fillOval`
4. 房屋：練習 `fillRect` 、 `drawRect`
5. 屋頂：練習 `drawLine`
6. 門與窗戶：練習圓角矩形和十字線

我們先從第一步開始：

```
// 天空
g.setColor(Color.CYAN);
g.fillRect(0, 0, 500, 250);

// 草地
g.setColor(Color.GREEN);
g.fillRect(0, 250, 500, 150);
```

這兩段分別代表：

- `fillRect(0, 0, 500, 250)` ：從左上角開始，畫一個寬500、高250的天空。
- `fillRect(0, 250, 500, 150)` ：從 Y=250 開始，畫出下半部草地。

先把這兩段放進 `paintComponent()` 執行看看。接下來我們再一起畫太陽，順便學會怎麼用 `for` 迴圈畫出 8 條光芒。

你說：

已選取 1 個片段

這個裡面怎麼填啊 數值

ChatGPT 說：

`g.fillRoundRect()` 跟 `g.fillRect()` 很像，只是它畫出來的矩形四個角是圓弧形的。

語法

```
g.fillRoundRect(x, y, width, height, arcWidth, arcHeight);
```

數值	意思
x	左上角 X 座標
y	左上角 Y 座標
width	矩形寬度
height	矩形高度
arcWidth	圓角的水平弧度大小
arcHeight	圓角的垂直弧度大小

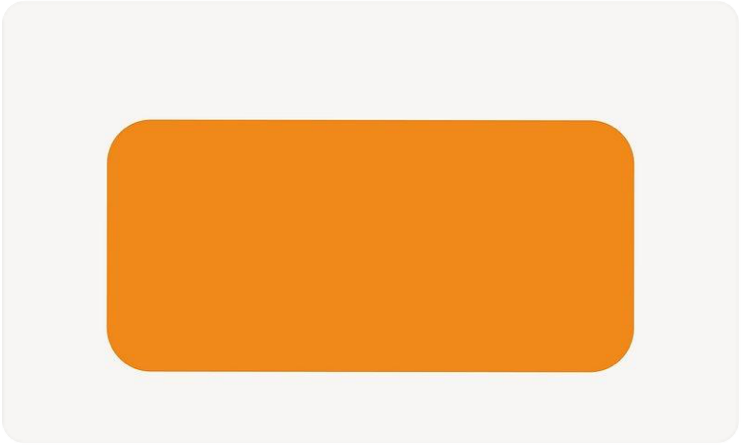
以你的房子門為例

```
// 門
g.setColor(Color.ORANGE);
g.fillRoundRect(240, 250, 45, 70, 10, 10);
```

代表：

- `240` ：門從 X=240 開始
- `250` ：門從 Y=250 開始
- `45` ：門的寬度
- `70` ：門的高度
- `10, 10` ：設定圓角大小

圓角大小比較



fillRoundRect(10,10)



fillRoundRect(40,40)

考試記憶： 前面四個數值決定矩形的位置和大小，最後兩個決定圓角大小。

你說：

```
import javax.swing.*;
import java.awt.*;

public class Main {
```

```
    public static void main(String[] args) {

        JFrame frame = new JFrame("小房子");

        MyPanel panel = new MyPanel();

        frame.add(panel);
        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

    }
}

class MyPanel extends JPanel {

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 500, 250);

        g.setColor(Color.GREEN);
        g.fillRect(0, 250, 500, 150);

        g.setColor(Color.RED);
        g.drawLine(100, 160, 400, 160);
        g.drawLine(250, 40, 100, 160);
        g.drawLine(250, 40, 400, 160);


        g.setColor(Color.ORANGE);
        g.fillRect(150, 160, 200, 150);
        g.setColor(Color.WHITE);
        g.fillRect(170, 180, 50, 50);
        g.fillRect(280, 180, 50, 50);
        g.setColor(Color.BLACK);
        g.drawRect(150, 160, 200, 150);
        g.drawRect(170, 180, 50, 50);
        g.drawRect(280, 180, 50, 50);
        g.drawLine(305, 180, 305, 230);
        g.drawLine(195, 180, 195, 230);
        g.drawLine(170, 205, 220, 205);
        g.drawLine(280, 205, 330, 205);

        g.setColor(Color.YELLOW);
        g.fillOval(400, 40, 80, 80);


        g.setColor(Color.WHITE);
        g.fillOval(60, 40, 100, 80);
        g.fillOval(10, 60, 100, 70);
        g.fillOval(110, 60, 100, 70);

        g.setColor(Color.DARK_GRAY);
        g.fillRoundRect(220, 240, 60, 70, 10, 10);
        g.setColor(Color.YELLOW);
        g.fillOval(260, 270, 10, 10);

        g.setColor(Color.BLACK);
        g.drawString("A1143308  林亞蒨", 370, 350);
```

}
} 我現在寫這樣 我的雲會被卡到 然後太陽的羨羨要怎麼畫

ChatGPT 說：

你目前的程式其實已經把房子、窗戶、門和背景完成了！現在主要有兩個地方要調整：

1. **雲朵被房子或其他圖形遮住。**
2. **太陽要加上至少 8 條光芒。**

我分開教你。

一、為什麼雲朵會被卡到？

因為 Java 繪圖有一個重要觀念：

後面畫的東西會蓋住前面畫的東西。

你目前的繪圖順序是：

```
// 先畫屋頂
g.drawLine(...);

// 再畫房子
g.fillRect(...);

// 最後才畫雲朵
g.fillOval(...);
```

所以你的雲朵其實會蓋在房子上面。

另外，你目前雲朵的三個圓形座標是：

```
g.fillOval(60, 40, 100, 80);
g.fillOval(10, 60, 100, 70);
g.fillOval(110, 60, 100, 70);
```

其中第一個和第三個會有部分超出房屋左側附近的範圍，但不會影響房子本身。

建議你把雲朵放到房子前面繪製，也就是先畫天空、草地、雲朵，再畫房子。

如果你希望雲朵在房子後面，建議把雲朵移到房屋之前畫。

二、太陽的光芒怎麼畫？

這題老師提示可以用 `for` 迴圈搭配 `Math.cos()` 、 `Math.sin()` 。

你的太陽目前是：

```
g.setColor(Color.YELLOW);
g.fillOval(400, 40, 80, 80);
```

所以太陽的中心點是：

- X = 440
- Y = 80
- 半徑 = 40

我們要讓光芒從太陽外圍往外延伸。

1. 加入這段程式

1. 加入這段程式

把它放在 `fillOval()` 之前：

```
g.setColor(Color.ORANGE);

for (int i = 0; i < 8; i++) {

    double angle = Math.toRadians(i * 45);

    int x1 = (int)(440 + Math.cos(angle) * 45);
    int y1 = (int)(80 + Math.sin(angle) * 45);

    int x2 = (int)(440 + Math.cos(angle) * 65);
    int y2 = (int)(80 + Math.sin(angle) * 65);

    g.drawLine(x1, y1, x2, y2);
}

g.setColor(Color.YELLOW);
g.fillOval(400, 40, 80, 80);
```

2. 這段程式在做什麼？

最重要的是這一行：

```
double angle = Math.toRadians(i * 45);
```

因為：

- `i = 0`：0 度
- `i = 1`：45 度
- `i = 2`：90 度
- `i = 3`：135 度
- …
- `i = 7`：315 度

總共 8 個方向，每個方向相差 45 度。

接著：

```
Math.cos(angle)
Math.sin(angle)
```

分別計算 X、Y 方向的位置。

而這兩組座標：

```
x1, y1
x2, y2
```

就是一條光芒的起點和終點。





三、你的程式還有一個小地方要注意

你的屋頂目前畫在房屋主體之前：

```
g.drawLine(100, 160, 400, 160);
g.drawLine(250, 40, 100, 160);
g.drawLine(250, 40, 400, 160);
```

但是後面畫房屋時：

```
g.fillRect(150, 160, 200, 150);
```

會把屋頂底下的部分蓋掉。

建議把屋頂的三條線移到房屋主體之後畫。

目前你已經練習到的重點：

- `fillRect()`：天空、草地、房屋、窗戶
- `drawRect()`：房屋與窗戶外框
- `fillOval()`：太陽、雲朵、門把
- `drawLine()`：屋頂、窗戶十字、太陽光芒
- `fillRoundRect()`：房門
- `for` + `Math.cos()` + `Math.sin()`：計算太陽光芒座標

這題其實很適合練習考試的座標觀念，尤其要記住：**先畫背景，再畫後面的景物，最後畫前面的物件。**

你說：

已選取 1 個片段

這兩個的用法是什麼樣

ChatGPT 說：

`Math.cos()` 和 `Math.sin()` 是 Java 用來計算三角函數的工具。

在你的作業中，它們主要用來**根據角度計算 X、Y 座標**，讓太陽的光芒可以往不同方向延伸。

一、基本用法

```
Math.cos(角度)
Math.sin(角度)
```

注意：Java 的 `cos()` 和 `sin()` 使用的是「弧度」，不是一般的角度。

所以如果你要使用 45 度，要先轉換：

```
double angle = Math.toRadians(45);

Math.cos(angle);
Math.sin(angle);
```

二、cos 和 sin 分別代表什麼？

可以先這樣記：

- `Math.cos(angle)`：計算水平方向的比例（X）。
- `Math.sin(angle)`：計算垂直方向的比例（Y）。

例如：

角度	cos	sin	方向
0°	1	0	右
90°	0	1	下
180°	-1	0	左
270°	0	-1	上

注意 Java 畫布的 Y 軸是往下增加，所以 90 度是往下。

三、搭配座標怎麼使用？

假設太陽中心在 `(440, 80)`，我們想畫一條往右的光芒。

```
int x = (int)(440 + Math.cos(0) * 50);
int y = (int)(80 + Math.sin(0) * 50);
```

計算結果：

- X = 440 + 1 × 50 = 490
- Y = 80 + 0 × 50 = 80

所以最後座標是 `(490, 80)`。

四、為什麼要搭配 for 迴圈？

因為我們想畫出 8 個不同方向的光芒：

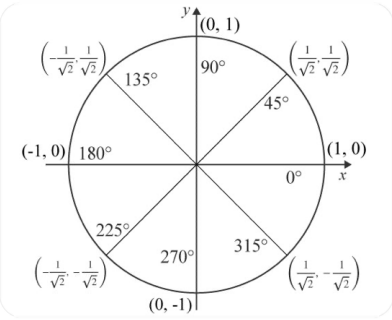
```
for (int i = 0; i < 8; i++) {

    double angle = Math.toRadians(i * 45);

    int x = (int)(440 + Math.cos(angle) * 50);
    int y = (int)(80 + Math.sin(angle) * 50);

}
```

每次迴圈都會換一個角度：



因此就能計算出八個不同方向的座標。

五、考試記憶重點

你可以直接記住這個公式：

```
int x = (int)(中心X + Math.cos(angle) * 半徑);
int y = (int)(中心Y + Math.sin(angle) * 半徑);
```

其中：

- 中心 X、Y：圖形的中心位置。
- `angle`：要朝向的角度。
- 半徑：距離中心多遠。
- `(int)`：將小數轉成整數，因為座標使用整數。

小練習：

假設太陽中心是 `(200, 200)`，角度是 90 度，半徑是 30。

請問最後算出來的 X、Y 座標是多少？

你可以試著算算看！

你說：

可以根據我的程式碼稍微去做修正嗎

ChatGPT 說：

可以！我會盡量保留你原本的程式碼、座標和畫圖方式，只修改幾個地方：

1. 把雲朵移到房子後面，避免被房子遮住。
2. 太陽增加 8 條光芒，使用你剛學的 `Math.cos()` 、 `Math.sin()` 。
3. 把屋頂移到房屋主體之後繪製，避免被房子蓋掉。
4. 保留你原本的房子、窗戶、門和顏色。

修改後的完整程式碼

```
import javax.swing.*;
import java.awt.*;

public class Main {

    public static void main(String[] args) {

        JFrame frame = new JFrame("小房子");

        MyPanel panel = new MyPanel();

        frame.add(panel);
        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);

    }
}

class MyPanel extends JPanel {
```

```
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        // 1. 天空與草地
        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 500, 250);

        g.setColor(Color.GREEN);
        g.fillRect(0, 250, 500, 150);


        // 2. 雲朵
        g.setColor(Color.WHITE);
        g.fillOval(60, 40, 100, 80);
        g.fillOval(10, 60, 100, 70);
        g.fillOval(110, 60, 100, 70);


        // 3. 太陽光芒
        g.setColor(Color.ORANGE);

        for (int i = 0; i < 8; i++) {

            double angle = Math.toRadians(i * 45);

            int x1 = (int)(440 + Math.cos(angle) * 45);
            int y1 = (int)(80 + Math.sin(angle) * 45);

            int x2 = (int)(440 + Math.cos(angle) * 65);
            int y2 = (int)(80 + Math.sin(angle) * 65);

            g.drawLine(x1, y1, x2, y2);
        }

        // 太陽本體
        g.setColor(Color.YELLOW);
        g.fillOval(400, 40, 80, 80);


        // 4. 房屋主體
        g.setColor(Color.ORANGE);
        g.fillRect(150, 160, 200, 150);

        g.setColor(Color.BLACK);
        g.drawRect(150, 160, 200, 150);


        // 5. 屋頂
        g.setColor(Color.RED);
        g.drawLine(100, 160, 400, 160);
        g.drawLine(250, 40, 100, 160);
        g.drawLine(250, 40, 400, 160);


        // 6. 窗戶
        g.setColor(Color.WHITE);
        g.fillRect(170, 180, 50, 50);
        g.fillRect(280, 180, 50, 50);
```

```
        g.setColor(Color.BLACK);
        g.drawRect(170, 180, 50, 50);
        g.drawRect(280, 180, 50, 50);

        g.drawLine(195, 180, 195, 230);
        g.drawLine(305, 180, 305, 230);
        g.drawLine(170, 205, 220, 205);
        g.drawLine(280, 205, 330, 205);


        // 7. 門
        g.setColor(Color.DARK_GRAY);
        g.fillRoundRect(220, 240, 60, 70, 10, 10);

        g.setColor(Color.YELLOW);
        g.fillOval(260, 270, 10, 10);


        // 8. 簽名
        g.setColor(Color.BLACK);
        g.drawString("A1143308 林亞蒨", 370, 350);

    }
}
```

這次最重要的修改

你可以特別理解這段：

```
double angle = Math.toRadians(i * 45);

int x1 = (int)(440 + Math.cos(angle) * 45);
int y1 = (int)(80 + Math.sin(angle) * 45);

int x2 = (int)(440 + Math.cos(angle) * 65);
int y2 = (int)(80 + Math.sin(angle) * 65);

g.drawLine(x1, y1, x2, y2);
```

- `(440, 80)`：太陽中心。
- `45`：光芒起點距離中心的距離。
- `65`：光芒終點距離中心的距離。
- `drawLine()`：把起點和終點連起來。

另外提醒你一個小地方： 你的太陽在 X=400、寬度80，因此右邊剛好到畫布邊界。光芒往右延伸時會超出畫布，這是正常的裁切現象。如果想讓光芒完整顯示，可以把太陽和光芒中心改成 `(420, 80)`。

我保留了你的原始座標，沒有大幅改動房屋設計，這樣你也比較容易理解自己的程式碼。

你說：

我想要雲跟屋子不要碰到 你可以幫我調整下嗎

ChatGPT 說：

可以！你原本的雲朵在左上方，但最下面會接近屋頂。我建議把雲朵整體往左上移一點，並稍微縮小，讓雲朵和屋頂有明顯間距。

你只需要修改原本的雲朵座標：

```
// 雲朵
g.setColor(Color.WHITE);
g.fillOval(35, 25, 80, 60);
g.fillOval(5, 45, 80, 55);
g.fillOval(75, 45, 80, 55);
```

這樣：

- 雲朵會集中在左上角。
- 三個白色橢圓仍然互相重疊。
- 雲朵最下方約在 Y=100。
- 屋頂最高點在 Y=40，但屋頂往左延伸的位置不會碰到雲朵。



其他程式碼都不用改，直接替換雲朵的三行就可以了！

你說：

整個程式碼謝謝

ChatGPT 說：

可以！這是幫你整合好的完整程式碼，保留你原本的房屋設計、顏色和座標，只調整雲朵位置與大小，讓雲朵和房子不要碰在一起。

```
import javax.swing.*;
import java.awt.*;

public class Main {

    public static void main(String[] args) {

        JFrame frame = new JFrame("小房子");

        MyPanel panel = new MyPanel();

        frame.add(panel);
        frame.setSize(500, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
```

```
    }
}

class MyPanel extends JPanel {

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        // 1. 天空與草地
        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 500, 250);

        g.setColor(Color.GREEN);
        g.fillRect(0, 250, 500, 150);


        // 2. 雲朵
        g.setColor(Color.WHITE);
        g.fillOval(35, 25, 80, 60);
        g.fillOval(5, 45, 80, 55);
        g.fillOval(75, 45, 80, 55);


        // 3. 太陽光芒
        g.setColor(Color.ORANGE);

        for (int i = 0; i < 8; i++) {

            double angle = Math.toRadians(i * 45);

            int x1 = (int)(420 + Math.cos(angle) * 45);
            int y1 = (int)(80 + Math.sin(angle) * 45);

            int x2 = (int)(420 + Math.cos(angle) * 65);
            int y2 = (int)(80 + Math.sin(angle) * 65);

            g.drawLine(x1, y1, x2, y2);
        }

        // 太陽本體
        g.setColor(Color.YELLOW);
        g.fillOval(380, 40, 80, 80);


        // 4. 房屋主體
        g.setColor(Color.ORANGE);
        g.fillRect(150, 160, 200, 150);

        g.setColor(Color.BLACK);
        g.drawRect(150, 160, 200, 150);


        // 5. 屋頂
        g.setColor(Color.RED);
        g.drawLine(100, 160, 400, 160);
        g.drawLine(250, 40, 100, 160);
        g.drawLine(250, 40, 400, 160);
```

```
        // 6. 窗戶
        g.setColor(Color.WHITE);
        g.fillRect(170, 180, 50, 50);
        g.fillRect(280, 180, 50, 50);

        g.setColor(Color.BLACK);
        g.drawRect(170, 180, 50, 50);
        g.drawRect(280, 180, 50, 50);

        g.drawLine(195, 180, 195, 230);
        g.drawLine(305, 180, 305, 230);
        g.drawLine(170, 205, 220, 205);
        g.drawLine(280, 205, 330, 205);


        // 7. 門
        g.setColor(Color.DARK_GRAY);
        g.fillRoundRect(220, 240, 60, 70, 10, 10);

        // 門把
        g.setColor(Color.YELLOW);
        g.fillOval(260, 270, 10, 10);


        // 8. 簽名
        g.setColor(Color.BLACK);
        g.drawString("A1143308 林亞蒨", 370, 350);

    }
}
```

這次調整的地方：

- 雲朵：移到左上方，縮小尺寸。
- 太陽：稍微往左移，讓光芒不會超出畫布。
- 房子：保留原本座標及設計。

另外，這份程式有符合老師要求的 8 個必要元素，可以作為你的作業版本。